

Electronics & Communication Systems

Hands-on Activity



Ing. Pietro Nannipieri – pietro.nannipieri@unipi.it
Ing. Tommaso Bocchi – tommaso.bocchi@phd.unipi.it
Prof. Luca Fanucci – luca.fanucci@unipi.it

Agenda

1. VHDL Practical Tips
2. VHDL Exercises

Agenda

1. VHDL Practical Tips
2. VHDL Exercises

VHDL Practical tips: Directory

- Directory structure
 - Project_name → Project directory, everywhere but avoid paths with spaces
 - src → Save here all your synthesizable .vhd source files. You can edit them on a generic text editor (Notepad++, Visual Studio Code, ...).
 - tb → Save here all your simulation-only .vhd files (test benches). You may have only one of them or more than one, anyway it is important to keep them separated from the source files.
 - modelsim → Directory for Modelsim Project. The tool writes the project file and all the auxiliary files only inside this directory.
 - vivado → Same as Modelsim, but for the Vivado tool.
 - Each project with its own directory!

Name	Date modified	Type	Size
 modelsim	08/04/2019 11:49	File folder	
 src	04/04/2019 11:31	File folder	
 tb	04/04/2019 11:31	File folder	
 vivado	23/10/2017 17:04	File folder	

VHDL Practical tips: File Name

- Entity and source file must have the same name.
 - No spaces
 - Can be case sensitive
- The name of the file and the architecture are used to link different entities in the library. Depending on the tool it may not be able to link the entity compiled in your library to its instances if the names do not match.

VHDL Practical tips: Port Map

- To recap, components port map can be done:

```
component fullAdder is
  port(-- Input of the full-adder
    a  : in std_logic;
    -- Input of the full-adder
    b  : in std_logic;
    -- Carry input
    c_i : in std_logic;
    -- Output of the full-adder
    o   : out std_logic;
    -- Carry output
    c_o : out std_logic
  );
end component;
```

Diagram illustrating two ways to map a component's ports:

Explicit (indicated by a yellow smiley face icon):

```
i_full_adder : fullAdder
  port map(
    a  => a(i),
    b  => b(i),
    c_i => c_int(i),
    o   => o(i),
    c_o => c_int(i + 1)
  );
```

Positional (indicated by a skull and crossbones icon):

```
i_full_adder : fullAdder
  port map(a(i), b(i), c_int(i), o(i), c_int(i + 1));
end generate;
```

- However, even if equivalent, **NEVER** use the positional. It can easily induce in error hard to debug.

VHDL Practical tips: Initialisation

- Initialisation of signals to be done only inside test benches

```
architecture bhv of counter_tb is -- Ts
    -----
    constant T_CLK    : time := 10 ns; -
    constant T_RESET  : time := 25 ns; -
    constant N_TB     : natural := 3;   -
    -----
    signal clk_tb : std_logic := '0'; -
    signal rst_tb  : std_logic := '0';
```

- No initialisation on VHDL source files (src/*):
 - It will be ignored by the synthesizer
 - It may initialise the circuit in a “specific” state
- To “initialise” internal signal, always assign default value in combinatory process and reset value in sequential process. Always assert a Reset at the beginning of every Testbench.

```
ddf_n_proc: process(clk, rst_n)
begin
    if(rst_n = '0') then
        q <= (others => '0');
    elsif(rising_edge(clk)) then
```

```
ddf_n_proc: process(a, b)
begin
    a <= '0'; --default
    b <= '1'; --default
```



To **avoid**
inferred
latches

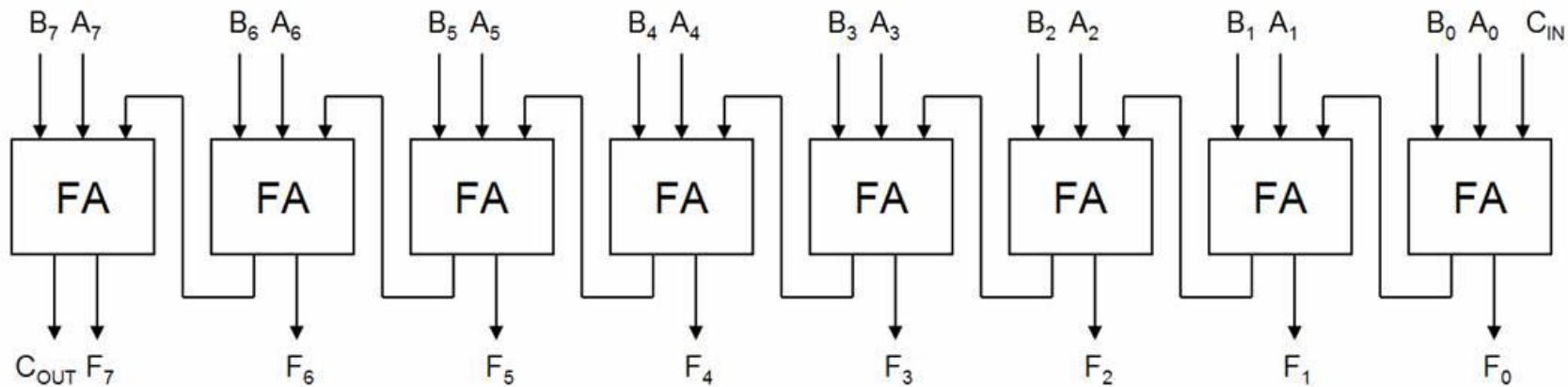
Agenda

1. Tool Installation
2. VHDL Practical Tips
3. VHDL Exercises

Ex 1 - Design and test of:

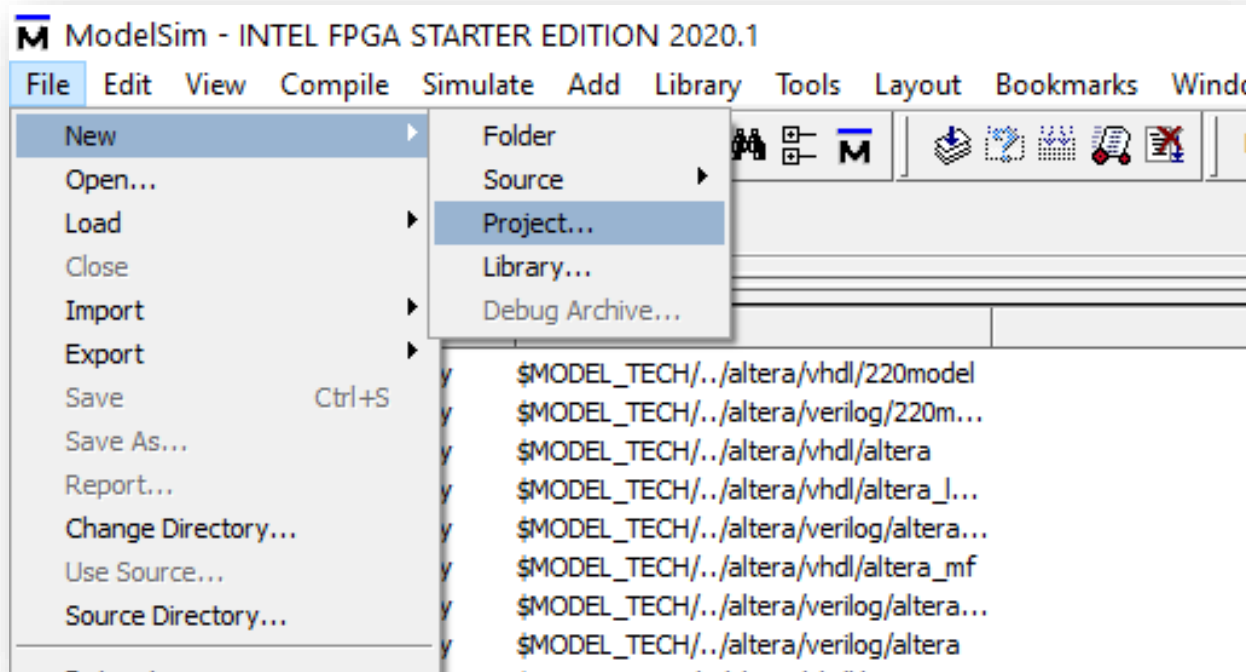
RCA - Ripple Carry Adder with configurable operands width (N) designed in structural way.

- HINTS:
 - for ... generate statement
 - FullAdder component



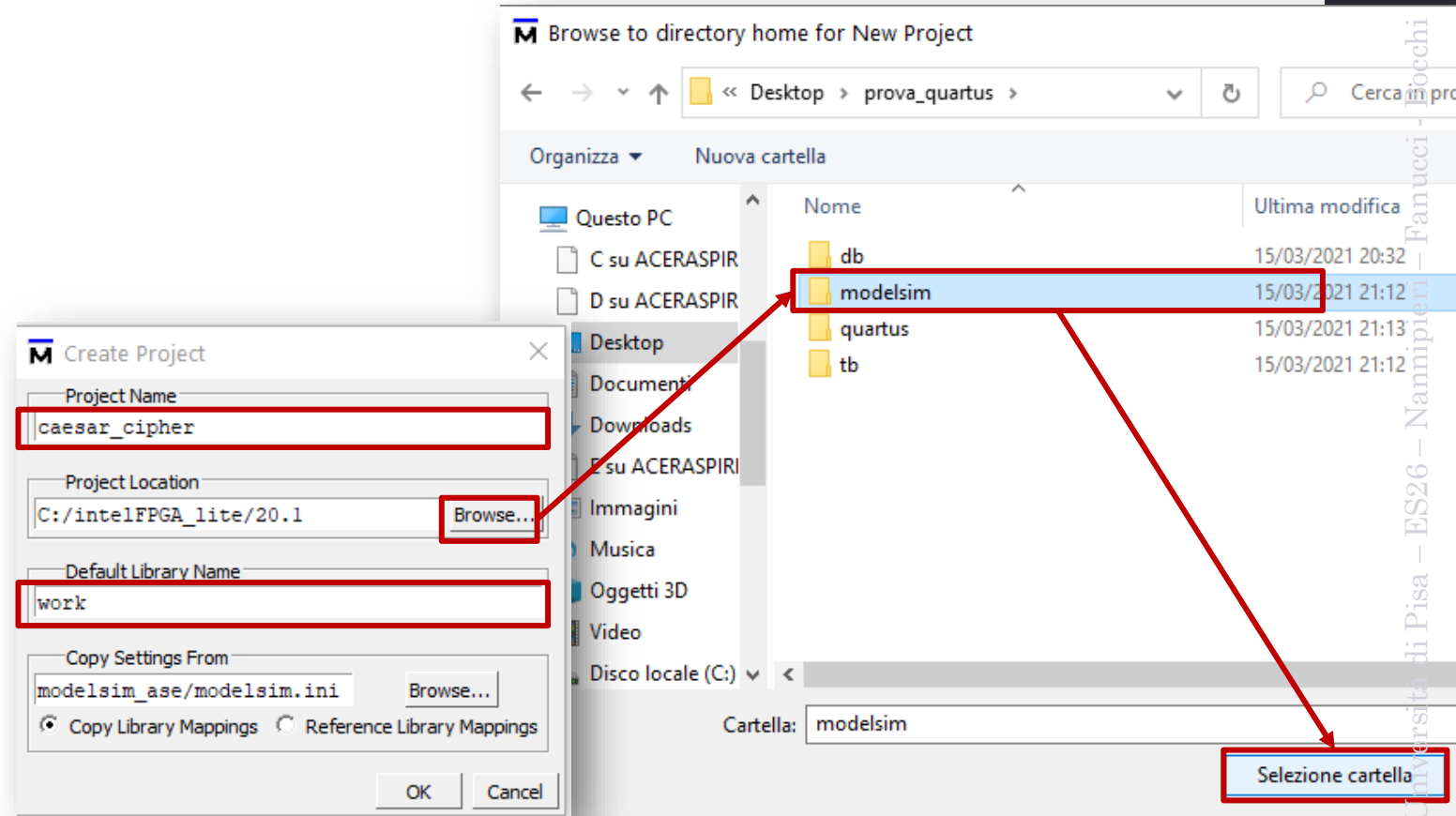
RTL Design in SV: Combinational logic

- Modelsim
 - Create a **New Project**



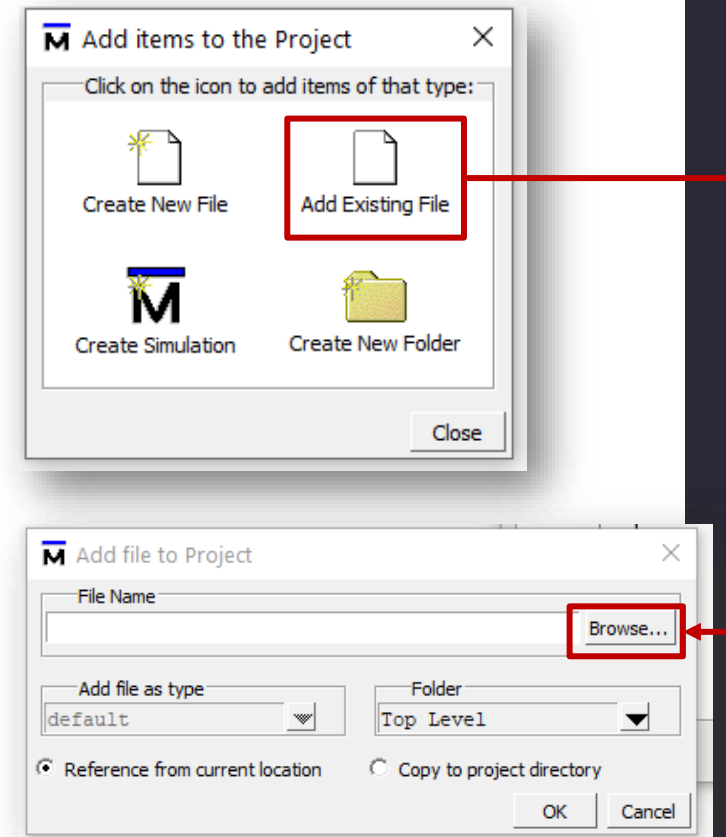
RTL Design in SV: Combinational logic

- Modelsim
 - Create a New Project
 - Project Name
 - Arbitrary name
 - Project Location
 - 'modelsim' sub-folder
 - Library Name
 - Leave 'work'



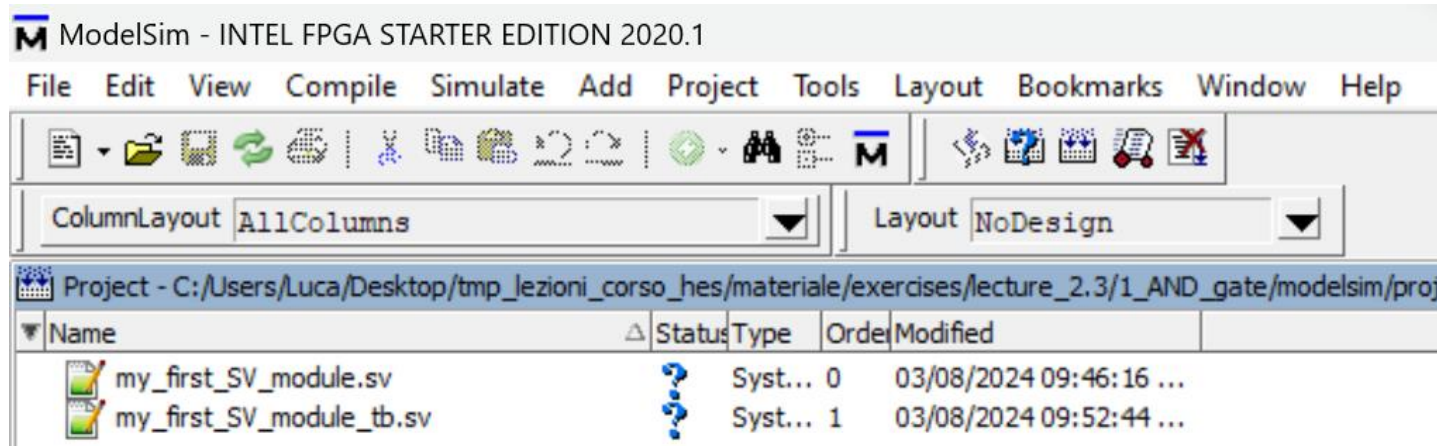
RTL Design in SV: Combinational logic

- Modelsim
 - Then, **Add Existing file**
 - Click on **Browse**, in the next dialog window
 - Select `../db/my_module.vhd`
 - Repeat again and select `../tb/my_tb.vhd`



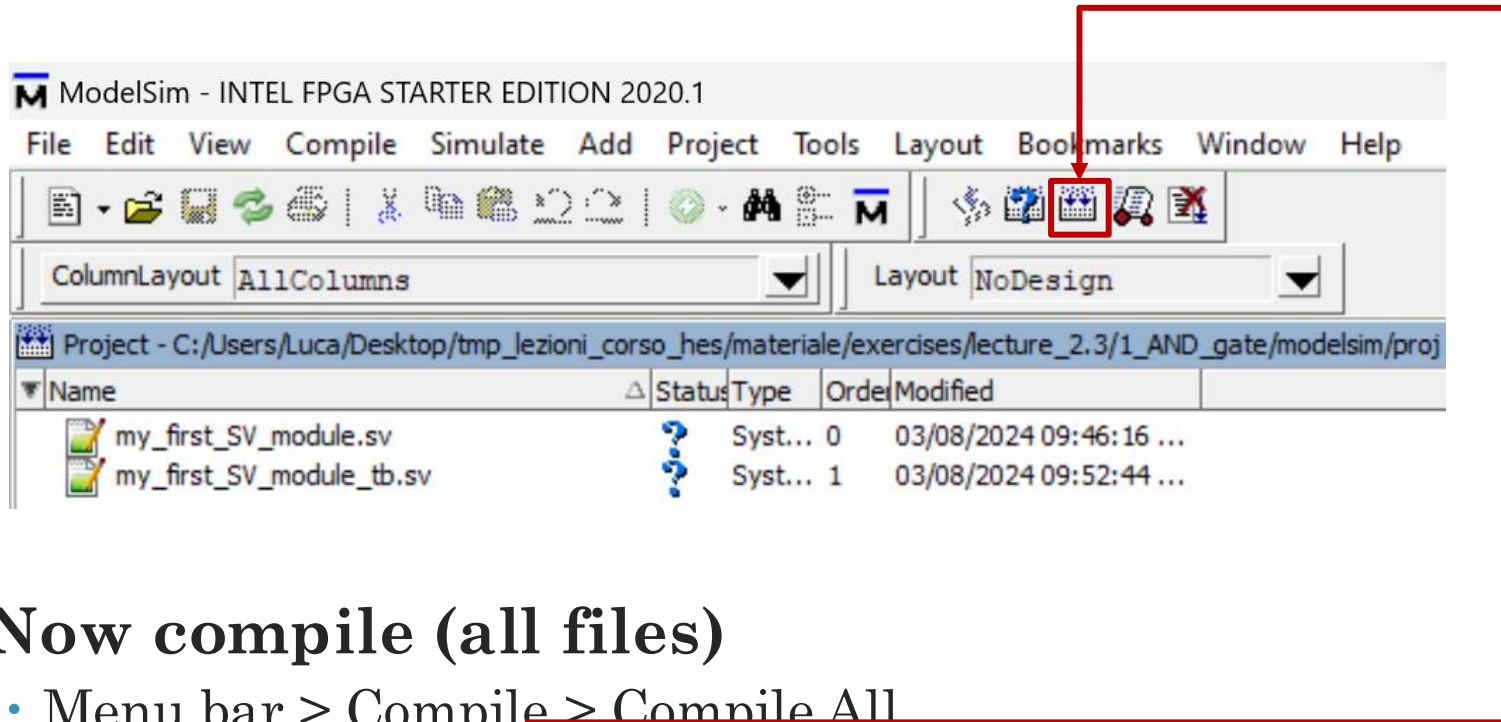
RTL Design in SV: Combinational logic

- Modelsim
 - You should see the following



RTL Design in SV: Combinational logic

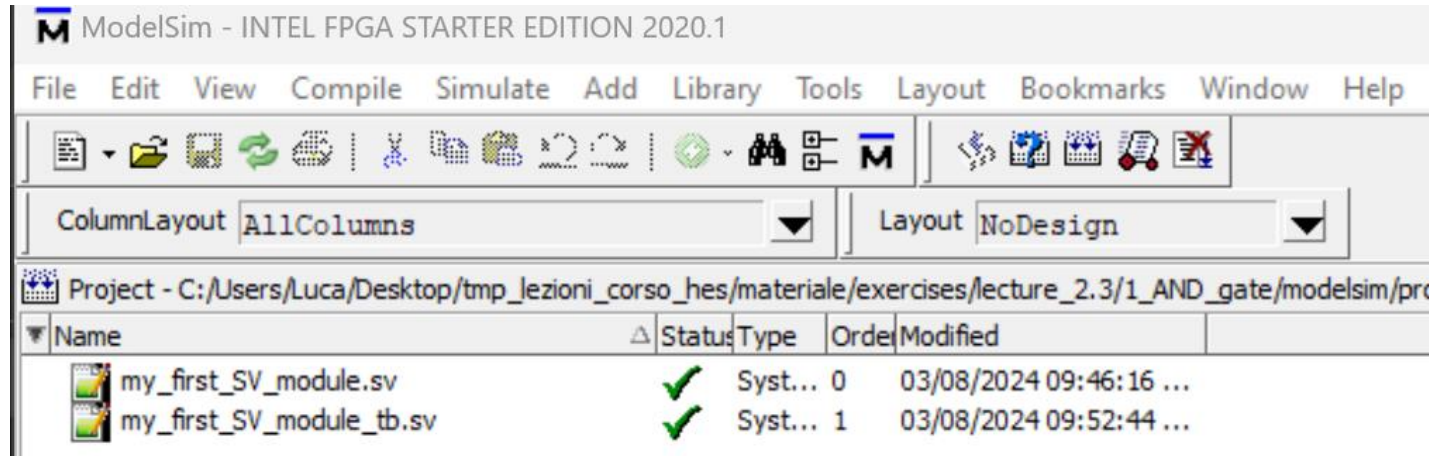
- Modelsim
 - You should see the following



- **Now compile (all files)**
 - Menu bar > Compile > Compile All
 - Or click this button

RTL Design in SV: Combinational logic

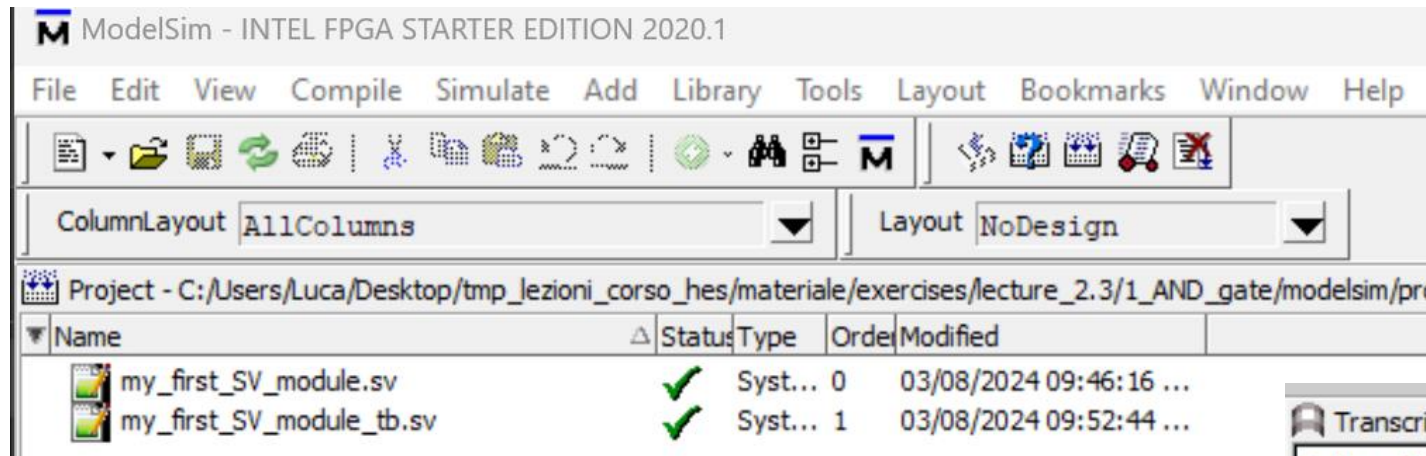
- Modelsim
 - You should see the following



- **Green tick** means “compilation successful”

RTL Design in SV: Combinational logic

- Modelsim
 - You should see the following



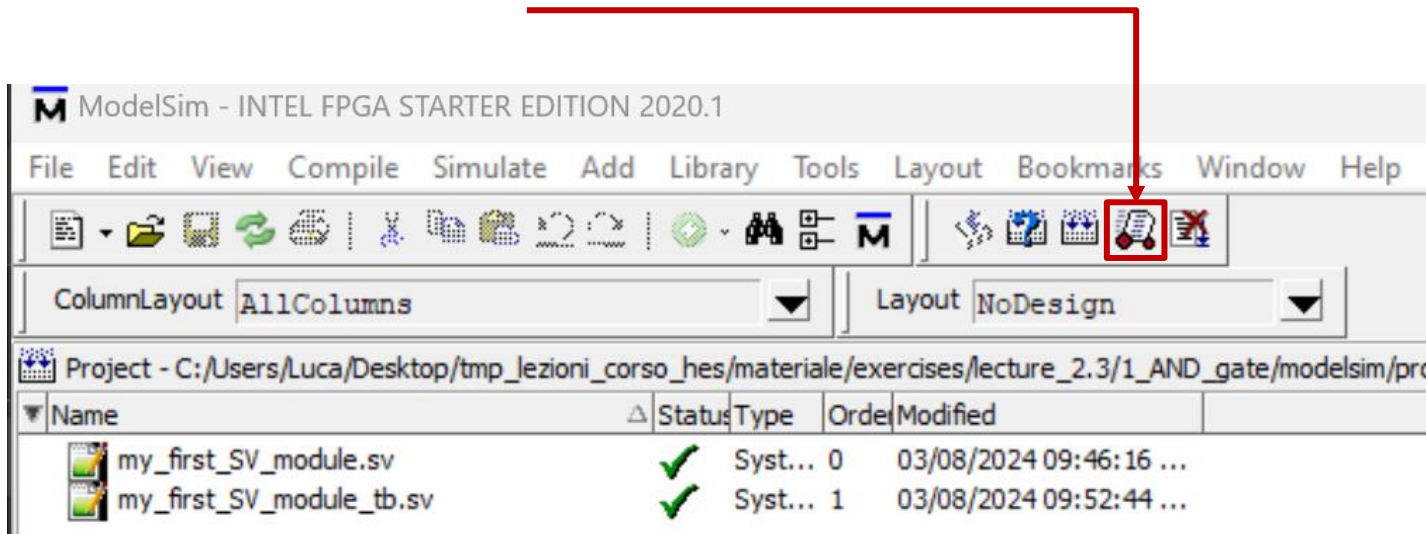
- Green tick means “compilation successful”, as also indicated in the **Transcript**

```
Transcript
# OpenFile C:/Users/Luca/Desktop/tmp_lezioni_corso_hes
# Loading project proj
# Compile of my_first_SV_module.sv was successful.
# Compile of my_first_SV_module_tb.sv was successful.
# 2 compiles, 0 failed with no errors.

ModelSim>
```

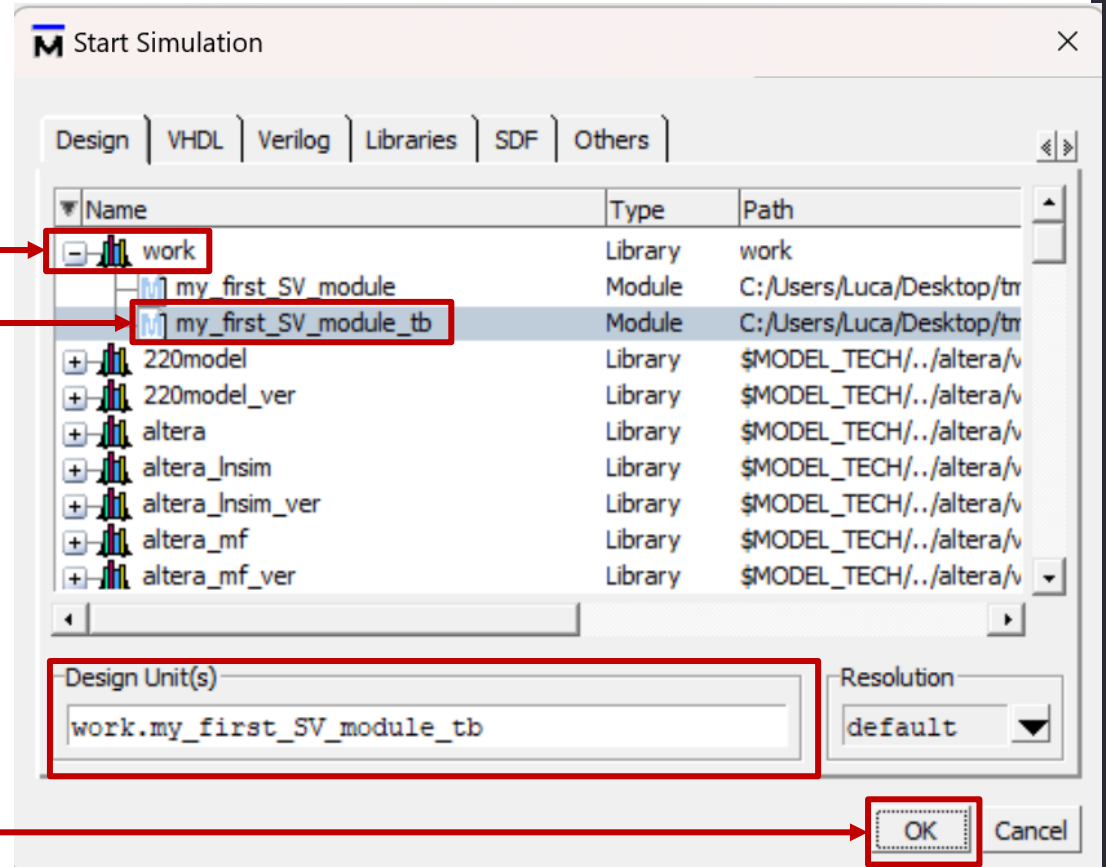
RTL Design in SV: Combinational logic

- Modelsim
 - **Now start the simulation**
 - Menu bar > Simulate > Start Simulation...
 - Or click this button



RTL Design in SV: Combinational logic

- Modelsim
 - **Now start the simulation**
 - A dialog window will open
 - List of all libraries
 - List of all design units for each library
 - Select the work library
 - Select the testbench unit
 - `my_first_SV_module_tb`
 - Then click OK



RTL Design in SV: Combinational logic

- Modelsim

- **Now run the simulation**

- Menu bar > Simulate > Run >
 - Run 100 → run for 100 time units
 - Run –All → run until end (when signals are never scheduled to change again)

- Or search this icon menu



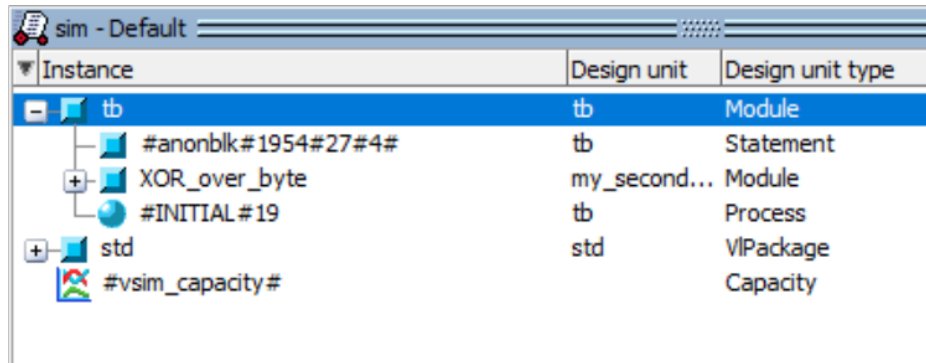
Edit to indicate the time units to simulate

Then click this button to run the simulation for that amount of time units

Or click this button for run all

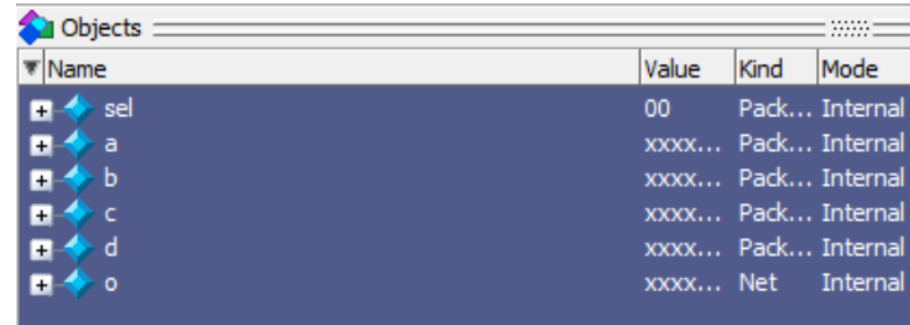
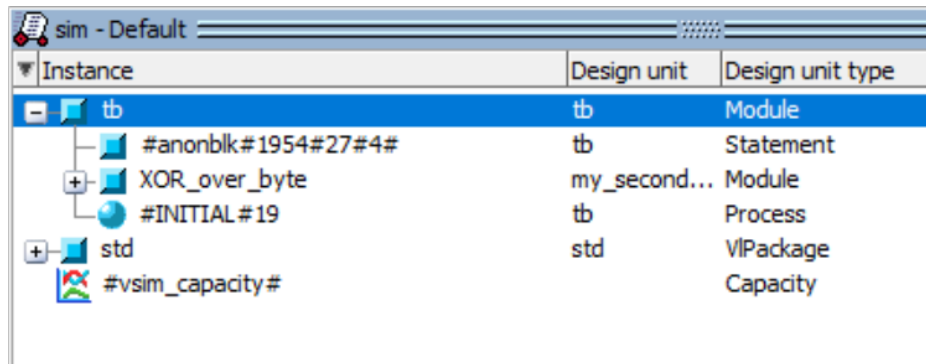
RTL Design in SV: Combinational logic

- Modelsim
 - Let's use waveforms
 - Before running the simulation
 - Select the testbench unit (tb) in the **Instance** window



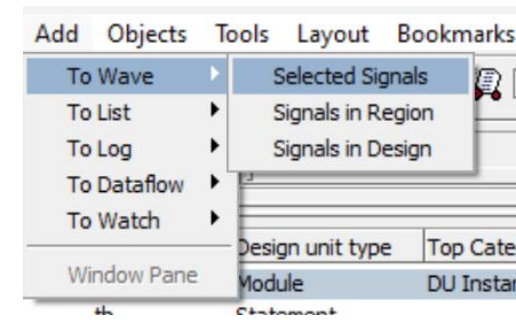
RTL Design in SV: Combinational logic

- Modelsim
 - Let's use waveforms
 - Before running the simulation
 - Select the testbench unit (tb) in the **Instance** window
 - You will see its corresponding signals in the **Objects** window



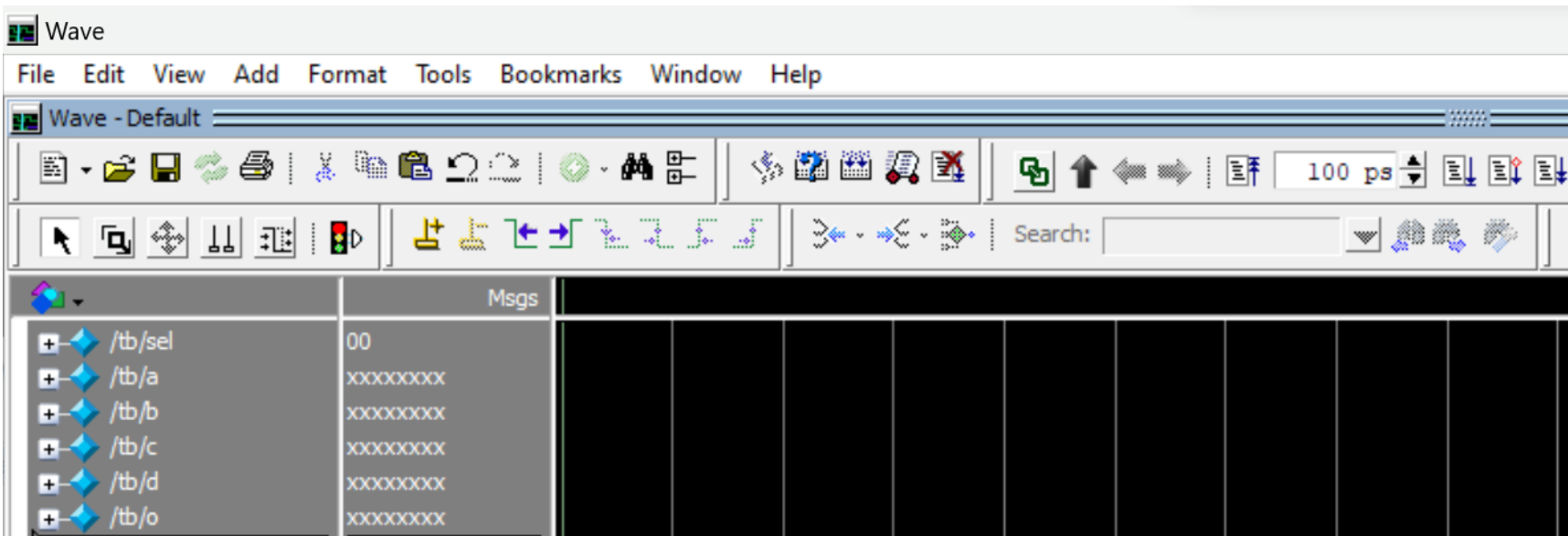
RTL Design in SV: Combinational logic

- Modelsim
 - Let's use waveforms
 - Before running the simulation
 - Select the testbench unit (tb) in the **Instance** window
 - Select the desired signals from the Objects window and then add them to a waveform
 - Menu bar > Add > To Wave > Selected Signals
 - Or, Ctrl + W



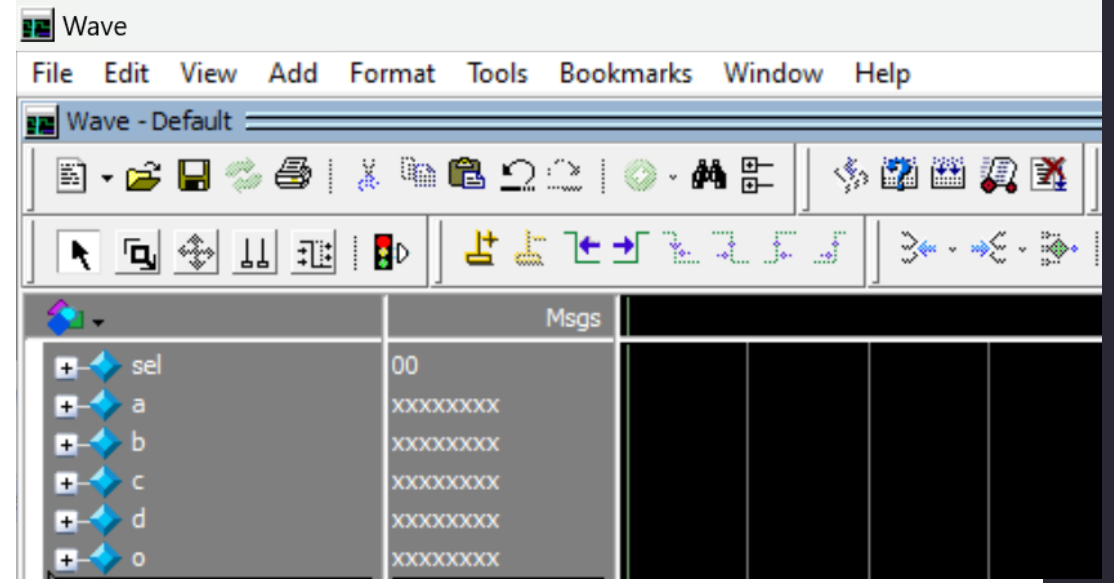
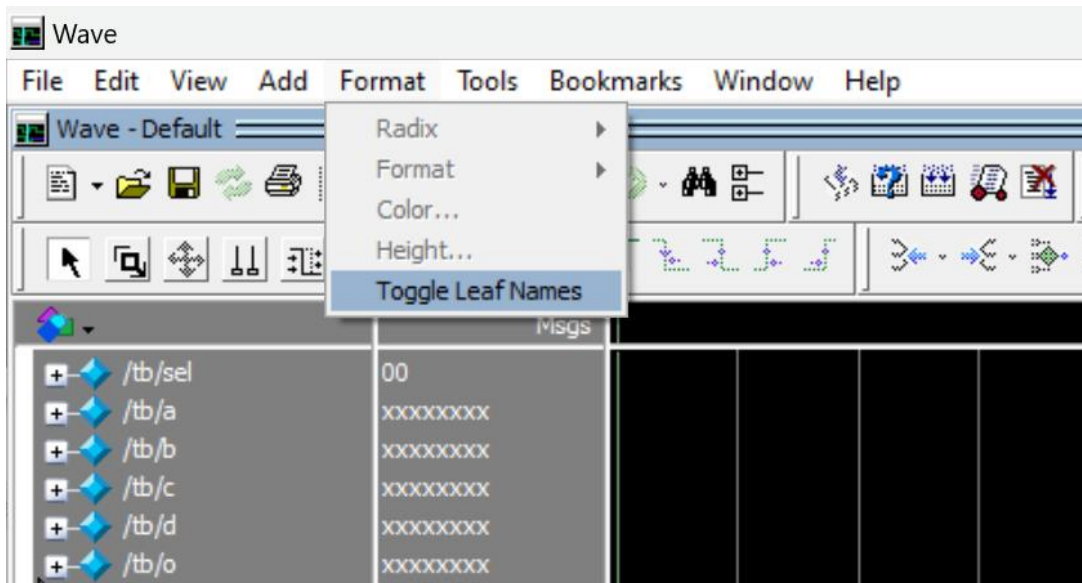
RTL Design in SV: Combinational logic

- Modelsim
 - A waveform window like the following one will open
 - Signals are reported with their hierarchical path



RTL Design in SV: Combinational logic

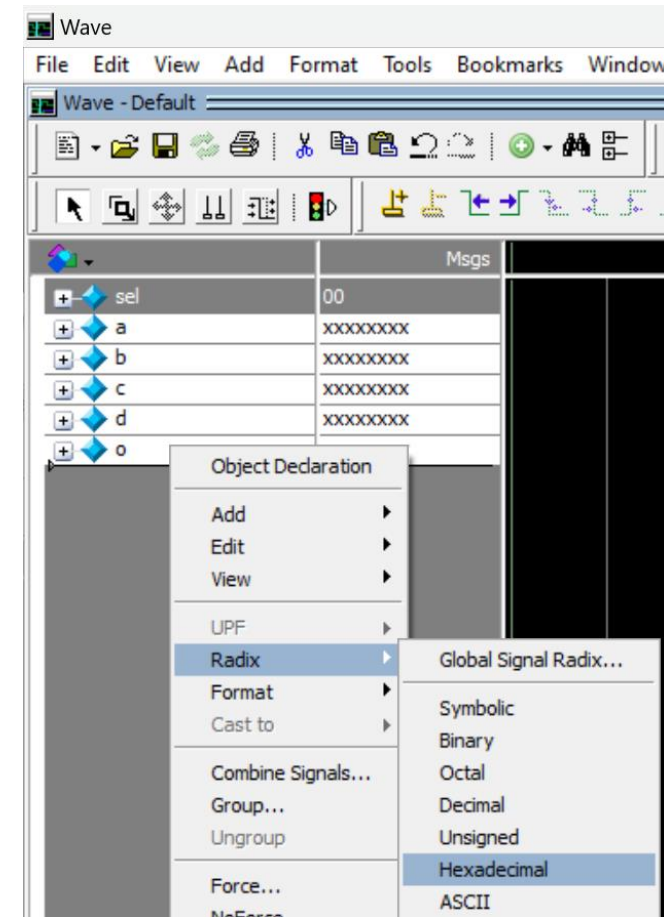
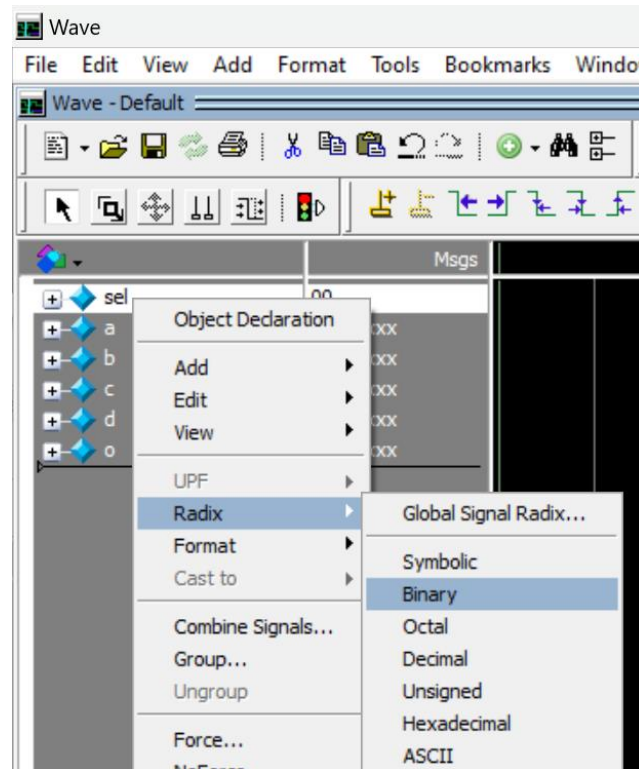
- Modelsim
 - A waveform window like the following one will open
 - Signals are reported with their hierarchical path
 - You can print just the signals name: Menu bar > Format > Toggle Leaf Names



RTL Design in SV: Combinational logic

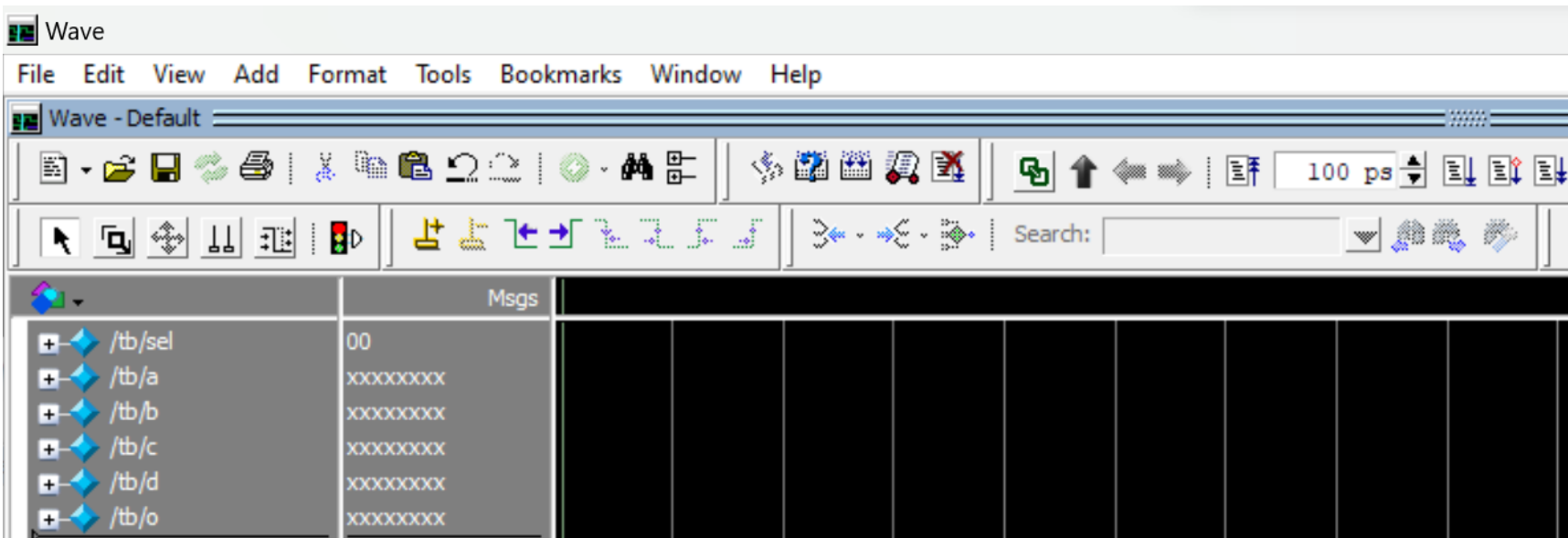
- Modelsim

- You can also change the display base of signals
 - Select the desired signal(s)
 - Right-click
 - Radix > 'select a base'



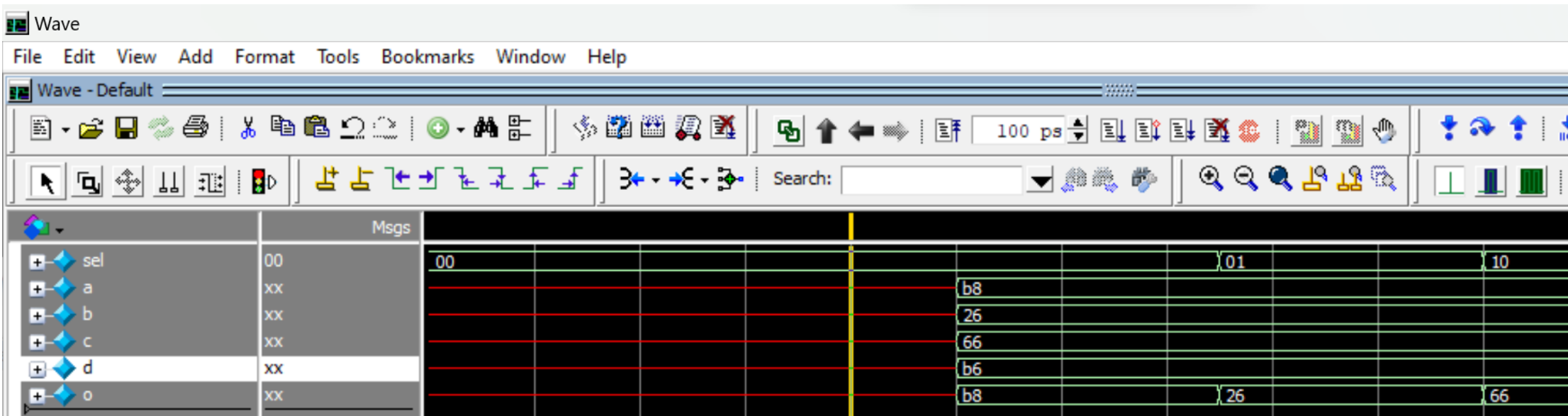
RTL Design in SV: Combinational logic

- Modelsim
 - Now you can run the simulation
 - Also the waveform window embeds the simulation control commands



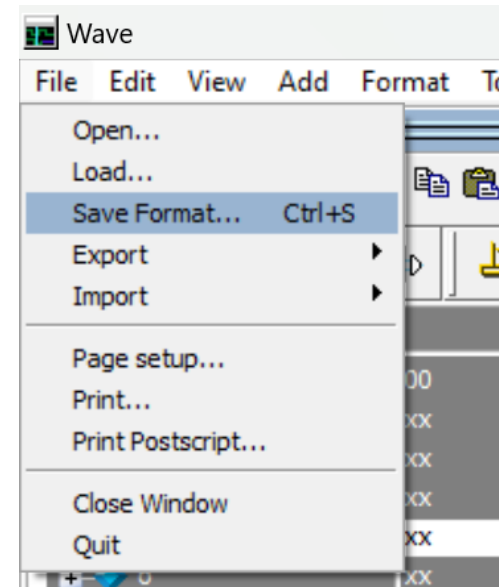
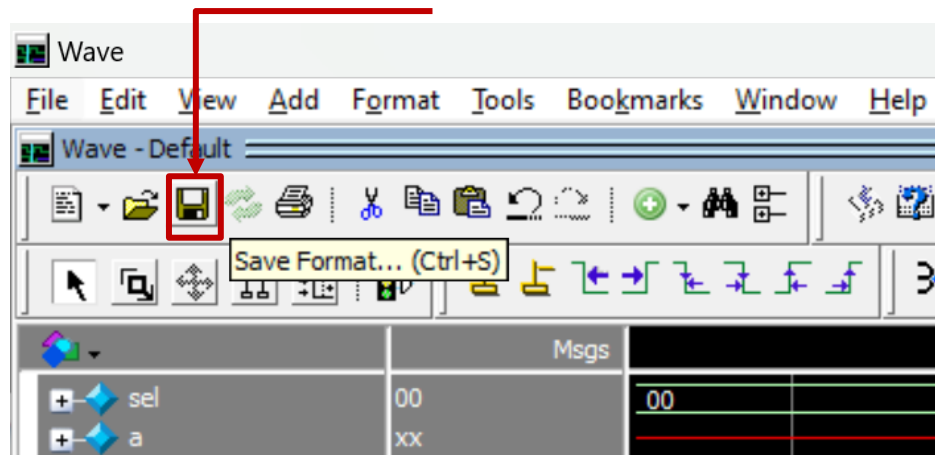
RTL Design in SV: Combinational logic

- Modelsim
 - Now you can run the simulation
 - Also the waveform window embeds the simulation control commands
 - If clicking on 'Run –All' ...



RTL Design in SV: Combinational logic

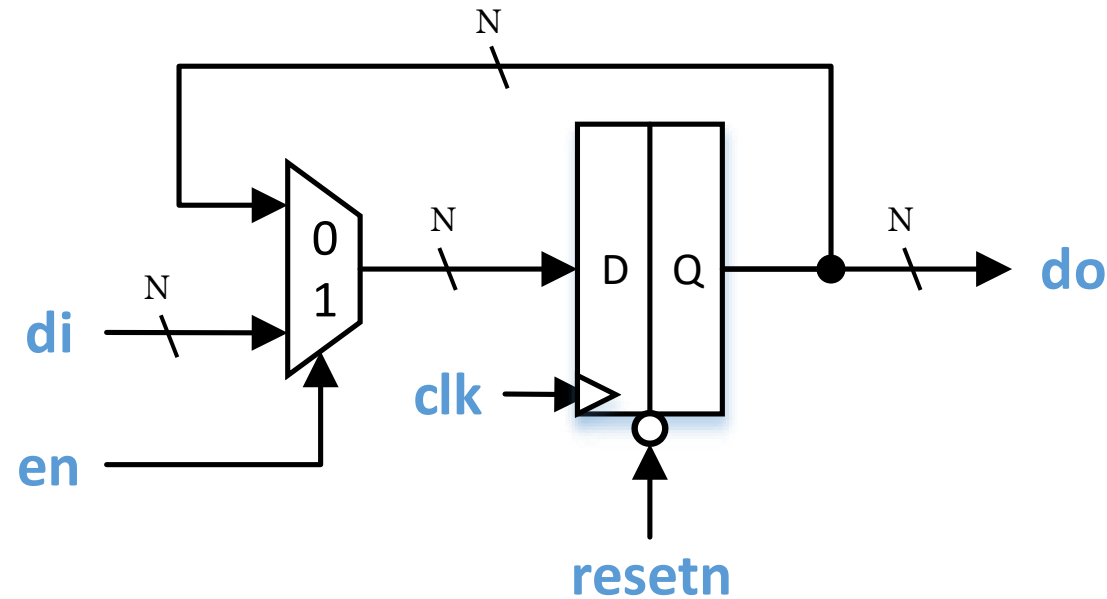
- Modelsim
 - Saving the waveform may be useful
 - For example, you have found a bug and want to rerun the simulation after updating the RTL code
 - Menu bar > File > Save Format...
 - Or, click this button



Ex 2 - Design and test of:

DFF_N - Parallel D Flip-Flop with configurable number of bits (N), asynchronous reset active low, with enable.

- HINTS:
 - `std_logic_vector` type

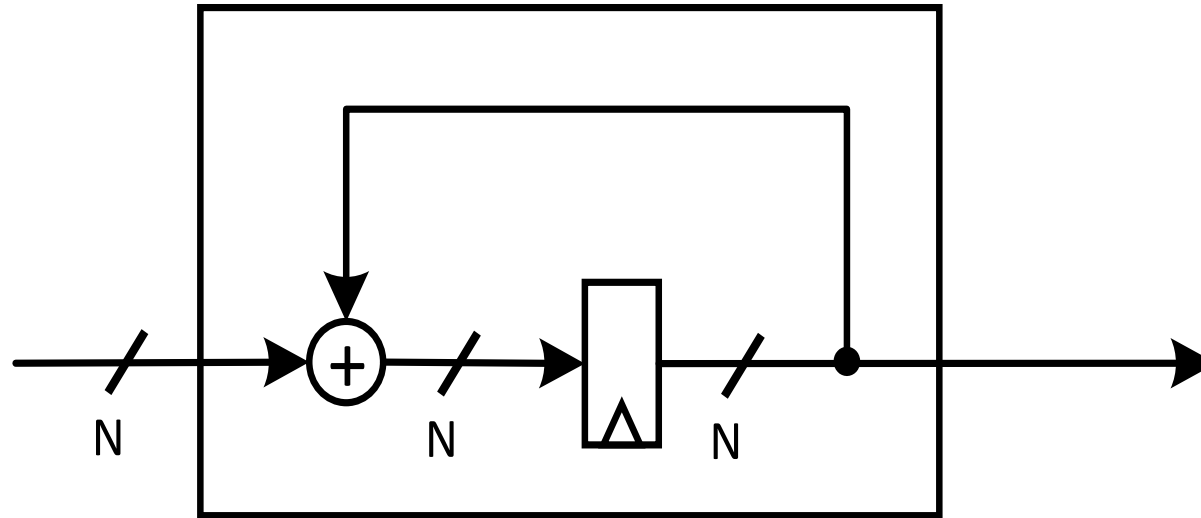


Ex 3 - Design and test of:

Counter: synchronous counter with configurable data width (N), asynchronous reset active low.

- HINTS:

- Ripple carry adder component
- DFF_N component



Ex 4 - Design and test of:

FIFO Buffer: First In First Out module with configurable depth (M) and data width (N), asynchronous reset active low.

- HINTS:
 - for ... generate statement
 - Two generics (M and N)
 - Use DDF_N component

